

Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
 - O-Notation, Ω -Notation, Θ -Notation
 - Fallanalyse: Worst Case, Best Case Analyse
- Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

Klassifikation der Wachstumsordnung

Wachstumsordnung : Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße basiert.

Description	O-Notation	Runtime if you double n	Example Algorithm
constant	$O(1)$	unchanged	Accessing an index of an array
logarithmic	$O(\log_2 n)$	increases slightly	Binary search
linear	$O(n)$	doubles	Looping over an array
log-linear	$O(n \log_2 n)$	slightly more than doubles	Merge sort algorithm
quadratic	$O(n^2)$	quadruples	Nested loops!
...	...	...	...
exponential	$O(2^n)$	multiplies drastically	Fibonacci with recursion

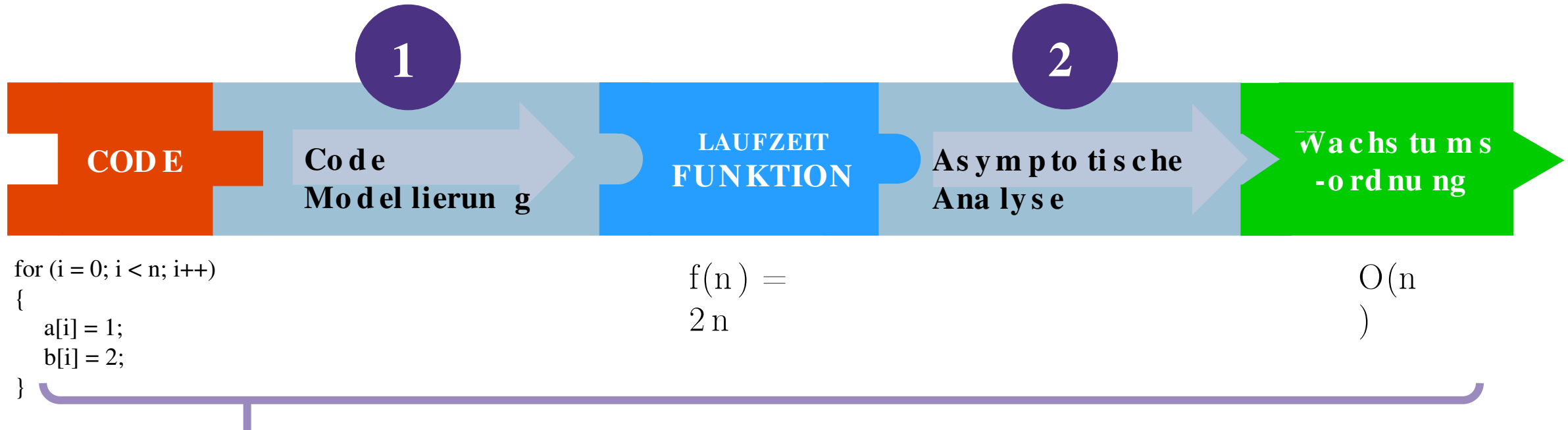
Code in O-Notation umwandeln



Bisher: “ $O(1)$ bedeutet keine Schleifen, $O(n)$ ist eine Schleife, $O(n^2)$ ist verschachtelte Schleife”

- Das ist weiterhin nützlich!
- In diesem Modul gehen wir mehr ins Detail: Damit wir auch komplizierteren Code in die O-Notation umwandeln können

Analyse von Algorithmen

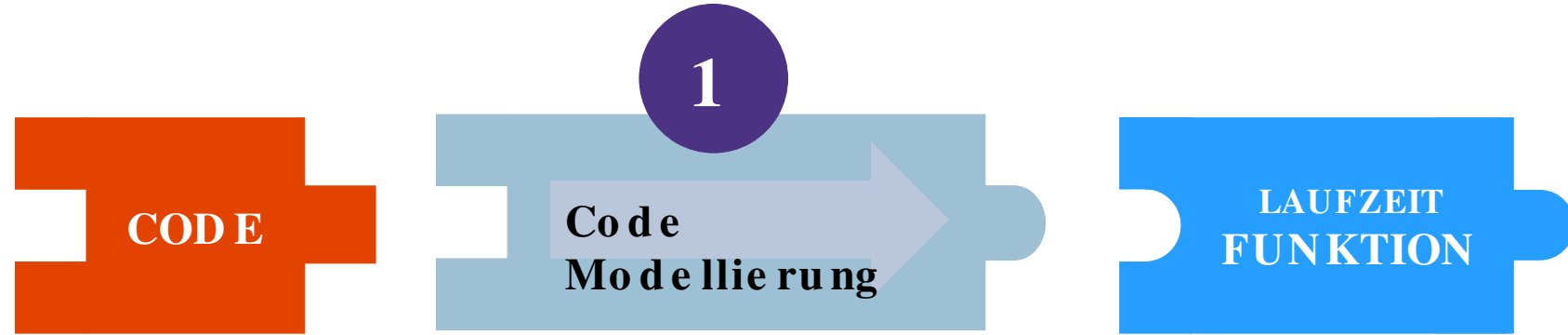


Analyse von Algorithmen: Der Gesamtprozess der Zuordnung von Code zu einer Wachstumsordnung, bestehend aus:

Code Modellierung: Code ~~[UWF010]~~ ^[ULF010] in Funktion, die die Laufzeit des Codes beschreibt

Asymptotische Notation: Funktion ~~[UWF010]~~ ^[UWF010] Wachstumsordnung, die das asymptotische Verhalten beschreibt

Code Modellierung



Code Modellierung - der Prozess der mathematischen Darstellung, wie viele Operationen ein Stück Code im Verhältnis zur Eingabegröße **n** ausführt.

- Umwandlung von Code in eine Laufzeitfunktion, die seine Laufzeit darstellt

CodeModellierung

Beispiel 2

```
public int method2(int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        int j = 0;
        while (j < n) {
            if (j % 2 == 0) {
                // do nothing
            }
            sum = sum + (i * 3) + j;
            j = j + 1;
        }
        i = i + 1;
    }
    return sum;
}
```

$$f(n) = (9n + 4)n + 3$$

Innere Schleife läuft n-mal

Äußere Schleife läuft n-mal

O-, Ω - und Θ -Notation

Übung

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$? $\Theta(n^2)$?

a

· $f(n) \in O(n^2)$

b

· $f(n) \in O(n^2)$

c

· $f(n) \in O(n^2)$

d

· $f(n) \in O(n^2)$ $f(n) \in \Omega(n^2)$ $f(n) \in \Theta(n^2)$

e

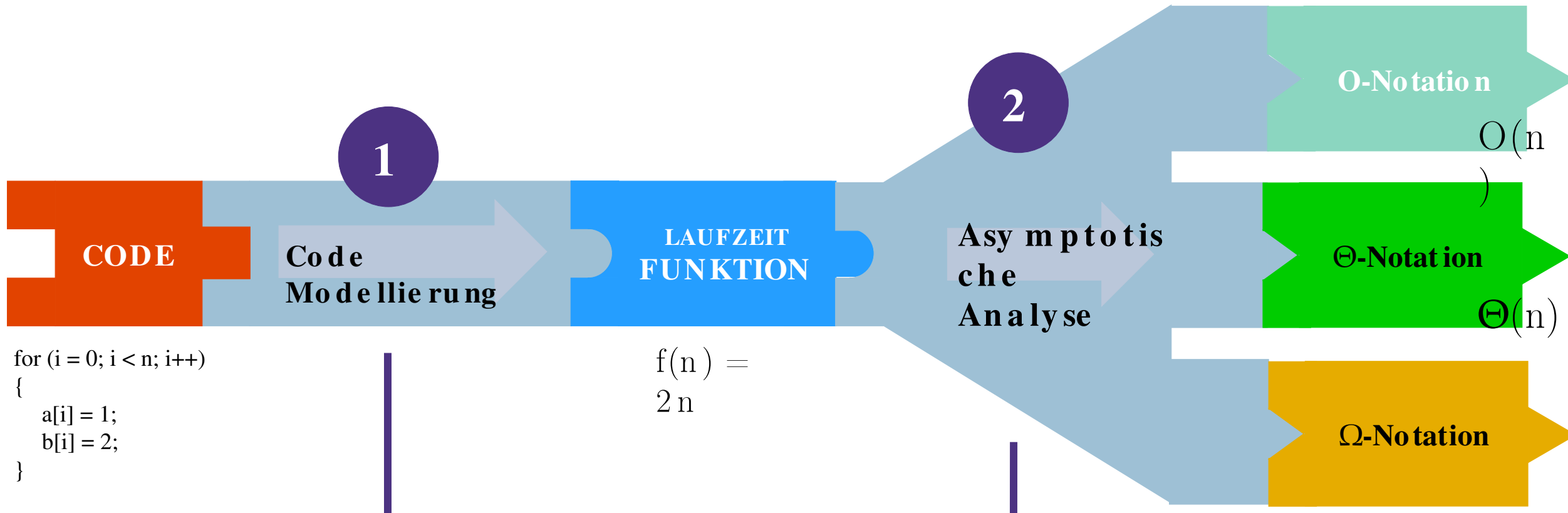
· $f(n) \in \Omega(n^2)$

O-Notation

Ω -Notation

Θ -Notation

Analyse von Algorithmen



Schauen wir uns dieses Werkzeug nun genauer an. Wie genau kommen wir zu dieser Funktion?

Wir haben gerade dieses Werkzeug fertiggestellt, um eine Funktion anhand einiger nützlicher Schranken zu charakterisieren!